Natural Language Processing
Assignment 1

Efe Özbal (s5996821)
Sigurdur Haukur Birgisson (s5982960)
Josh van Vliet (s5950465)

February 20, 2026

1 Dataset & Setup

Dataset for this text classification by topic task is the textual AG News dataset from Hugging Face [**?**]. Each data point has an associated label, the news title and its description. There are 4 classes, all of which has 30000 training and 1900 testing samples: world, sports, business, and sci/tech.

We use the official Hugging Face split (120,000 train and 7,600 test). We use stratified sampling (90/10) with seed 67 to create a development set:

- Train: 108,000
- Dev: 12,000
- Test: 7,600

1.1 Preprocessing

The input is the title and description concatenated. We used the following preprocessing steps:

- Lowercasing
- Whitespace removal
- Punctuation removal
- English stopword removal (using NLTK library)
- Lemmatization (with WordNetLemmatizer)

The reason why we remove stop words, is because common words often carry less information and carry noise into our classifier.

The last step in the normalization is lemmatization, the act of reducing words into their dictionary form (e.g. "running" to "run"). Using the NLTK's WordNetLemmatizer, we reduced each word in the text to their dictionary forms to reduce the computational complexity. In this way, different forms of the same word are reduced to a single form.

1.2 Tokenization

Even though there is no explicit tokenization in our code, texts are vectorized with word-level TF-IDF. After the normalization steps, TF-IDF vectorizes, and thus tokenizes, the input text into numerical data:

- n-gram range: (1, 2)
- `max_features` = 10,000
- `min_df` = 2

1.3 Reproducibility

All experiments make use of the fixed random seed 67. We use it for the data split and model initialization where it is applicable.

2 Models

In this section, we describe the two models we implemented for the text classification task:

- TF-IDF + Logistic Regression
- TF-IDF + LinearSVM.

Both models utilize Term Frequency-Inverse Document Frequency (TF-IDF) for feature extraction, which transforms the textual data into a numerical representation based on the importance of words in the documents.

The only difference between them is in the linear classifier head.

2.1 Hyperparameter Search

Model selection was done only on the dev split.

2.1.1 Logistic Regression Grid

- $C \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$
- `max_iter` = 1,000

2.1.2 Linear SVM Grid

- $C \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$
- loss $\in$ {hinge, squared_hinge}

2.1.3 Best Dev Split Settings

The best settings selected on the dev split were:

- Logistic Regression: $C = 1.0$ (Dev Accuracy: 0.9145)
- Linear SVM: $C = 1.0$, loss=hinge (Dev Accuracy: 0.9163)

The model architecture is shown in Figure **??**. The input text is first preprocessed and then transformed into TF-IDF features. These features are then fed into either a Logistic Regression classifier or a Linear Support Vector Machine (SVM) for classification.

3 Experimental Protocol

In this section, we describe the experimental protocol we followed for tuning our models and selecting the best hyperparameters.

3.1 Model Selection Protocol

We followed this procedure for model selection:

1. Build vocabulary for TF-IDF on training split only.
2. Train each model on train/dev splits.
3. Select the best hyperparameters based on our dev split accuracy.
4. Do a final evaluation on the test split for final results.

This protocal ensures that we do not have any data leakage from our test data into our model selection.

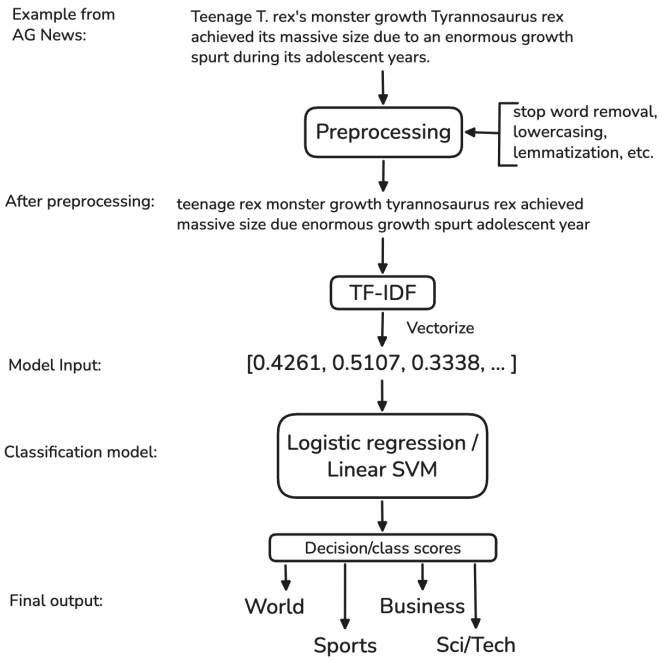

Figure 1: Model architecture for TF-IDF + Logistic Regression and TF-IDF + LinearSVM. The figure shows a random sample from the training set, which is preprocessed and transformed into TF-IDF features before being fed into the classifiers.

3.2 Training and Reproducibility

We set max_iter = 1,000 for Logistic Regression and Linear SVM as early stopping. This is to prevent unnecessary optimization steps and promote convergence.

During training, we kept the random seed at 67 (same as for data splitting) for reproducibility purposes.

3.3 Hyperparameter Tuning

Hyperparameters were tuned with grid search on just the dev split. The exact details are defined in Section 2.

3.4 Evaluation and Fair Comparison

For each model, we look at:

- Accuracy

- Macro-F1
- Confusion matrix

Macro-F1 is used to show performance for all four classes.

To make sure there is a fair comparison between the models, both have:

- The same train/dev/test splits
- The same preprocessing and TF-IDF feature settings
- The same random seed (67)
- Tuning only on the dev split and a final test eval

4 Results

In this section, we present the results of our experiments on the test set for both the Linear SVM and Logistic Regression models. We report the classification reports (precision, recall, F1-score, and support) for each class, as well as the confusion matrices for both models. Additionally, we provide a comparison of the overall accuracy and macro-F1 scores for both models on the test set.

In Table **??**, we can see that the Linear SVM model achieved an overall accuracy of 0.92 and a macro-F1 score of 0.92 on the test set. The Logistic Regression model, as shown in the subsequent table (Table **??**), achieved an overall accuracy of 0.91 and a macro-F1 score of 0.91 on the test set.

In the confusion matrices (Figures **??** and **??**), we can observe that both models have similar patterns of misclassification, with the most common confusion occurring between the sci/tech and business classes. This is reflected in the classification reports where the precision, recall, and F1-scores for these classes are lower compared to the sport and world classes. Overall, the Linear SVM model performs slightly better than the Logistic Regression model on both accuracy and macro-F1, but the difference is not very large (e.g., 0.01 in accuracy and macro-F1).

Comparing the two models in Table **??**, we can see that the Linear SVM model has a slightly higher accuracy and macro-F1 score compared to the Logistic Regression model on the test set. However, both models perform quite well, with accuracies and macro-F1 scores above 0.90, indicating that the TF-IDF features are effective for this classification task.

	Precision	Recall	F1-Score	Support
1	0.94	0.90	0.92	1900
2	0.95	0.98	0.97	1900
3	0.89	0.88	0.89	1900
4	0.89	0.90	0.90	1900
Accuracy			0.92	7600
Macro avg	0.92	0.92	0.92	7600
Weighted avg	0.92	0.92	0.92	7600

Table 1: Classification Report for Linear SVM on Test Set

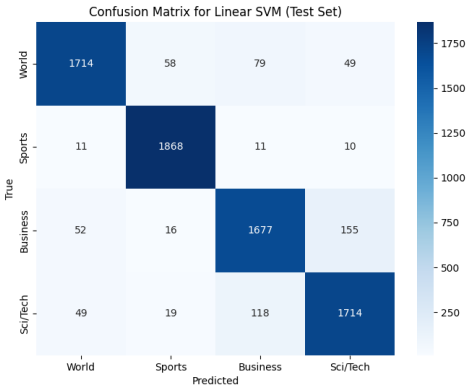

Figure 2: Confusion Matrix for Linear SVM on Test Set

	Precision	Recall	F1-Score	Support
1	0.93	0.91	0.92	1900
2	0.95	0.98	0.96	1900
3	0.88	0.89	0.88	1900
4	0.89	0.89	0.89	1900
Accuracy			0.91	7600
Macro avg	0.91	0.91	0.91	7600
Weighted avg	0.91	0.91	0.91	7600

Table 2: Classification Report for Logistic Regression on Test Set

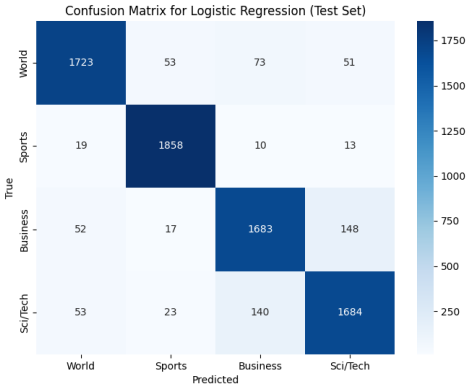

Figure 3: Confusion Matrix for Logistic Regression on Test Set

Model	**Accuracy**	**Macro-F1**
Linear SVM	**0.92**	**0.92**
Logistic Regression	0.91	0.91

Table 3: Comparison of Accuracy and Macro-F1 for Linear SVM and Logistic Regression on Test Set

5 Error Analysis

In this section, we will analyze the errors made by our best-performing models (Linear SVM and Logistic Regression) on the test set. We will look at the confusion matrices and t-SNE projections of the predictions to understand which classes are most often confused and why.

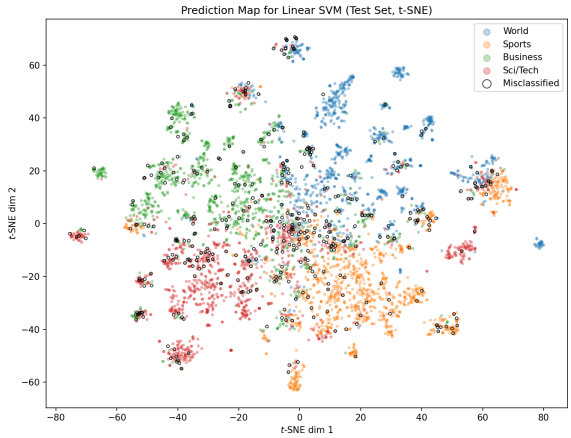

Figure 4: t-SNE Projection of Linear SVM Predictions on Test Set. Correctly classified samples are colored according to their true class, while misclassified samples are outlined in black.

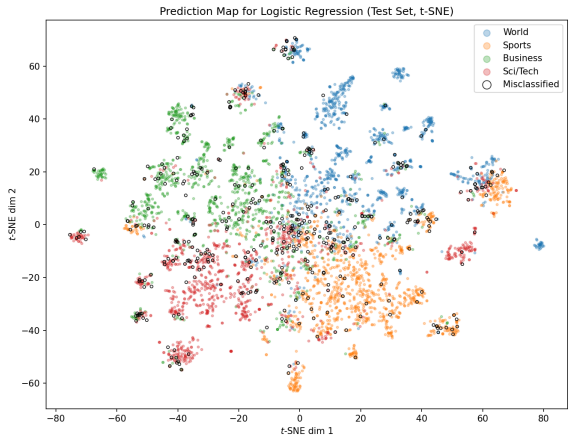

Figure 5: t-SNE Projection of Logistic Regression Predictions on Test Set. Correctly classified samples are colored according to their true class, while misclassified samples are outlined in black.

6 Discussion

7 Limitations

A Appendix